

C++ PROJECT

BANKING SYSTEM

A Console-Based Banking Application using Object-Oriented C++

01 | ABSTRACT

Overview

This project presents a console-based Banking System built in C++ using Object-Oriented Programming (OOP). It simulates real banking operations and is version-controlled on GitHub.

Core Focus

The system is structured around four classes — Account, User, Bank, and main() — each with a distinct responsibility. STL vectors handle dynamic in-memory data storage.

Key Concepts

Encapsulation, modular design, dynamic memory, and input validation are the core engineering principles demonstrated.

C++

Language

Dev-C++

IDE

OOP

Paradigm

GitHub

Version Control

02 | INTRODUCTION

A Banking System is a critical software application that manages financial data, ensures transaction integrity, and provides secure access to user accounts. This C++ project demonstrates these concepts in a simulated console environment.

01 OOP Design

Apply classes, encapsulation & dynamic memory management in C++.

02 Core Features

Deposit, Withdraw, Transfer & Transaction History operations.

03 GitHub Workflow

Feature-branch strategy, Pull Requests & clean commit history.

Technologies Used:

C++

Dev-C++ 5.11

STL Vectors

GitHub

OOP Classes

g++ Compiler

03 | ALGORITHM

Register

- Input username & password
- Check if username exists
- If not → Create User + Account
- Assign account number (1001+)
- Save to users list

Login

- Input username & password
- Loop through all users
- Match both credentials
- On success → load Banking Menu
- On fail → show error

Deposit

- Input amount
- Check amount > 0
- balance += amount
- Save to transaction history
- Print confirmation

Withdraw

- Input amount
- Check amount > 0
- Check amount <= balance
- balance -= amount
- Save to history

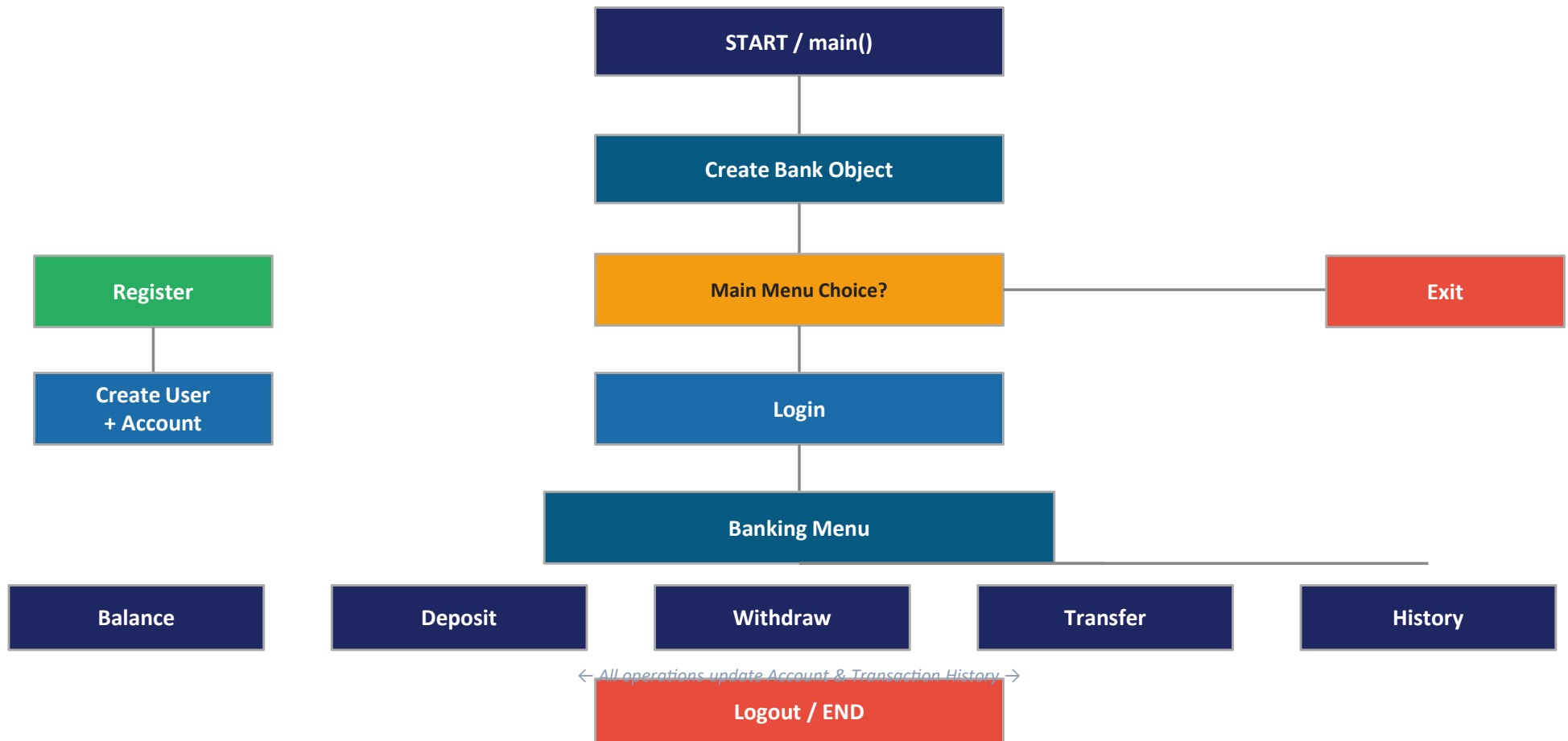
Transfer

- Input receiver acc + amount
- Search receiver in users[]
- Verify sender balance
- Sender withdraw, receiver deposit
- Update both histories

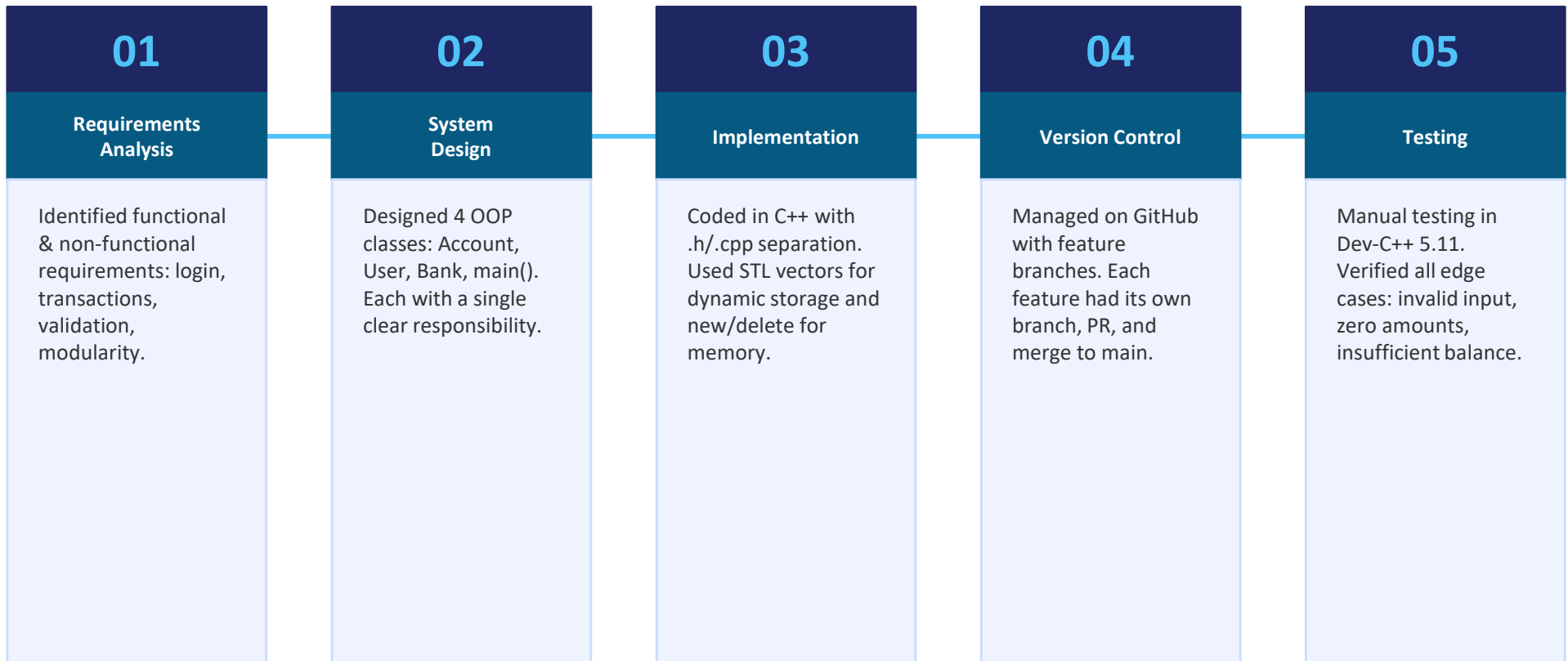
Exit

- User selects Exit
- Loop ends
- Delete all User pointers
- Free allocated memory
- Program terminates

04 | FLOWCHART

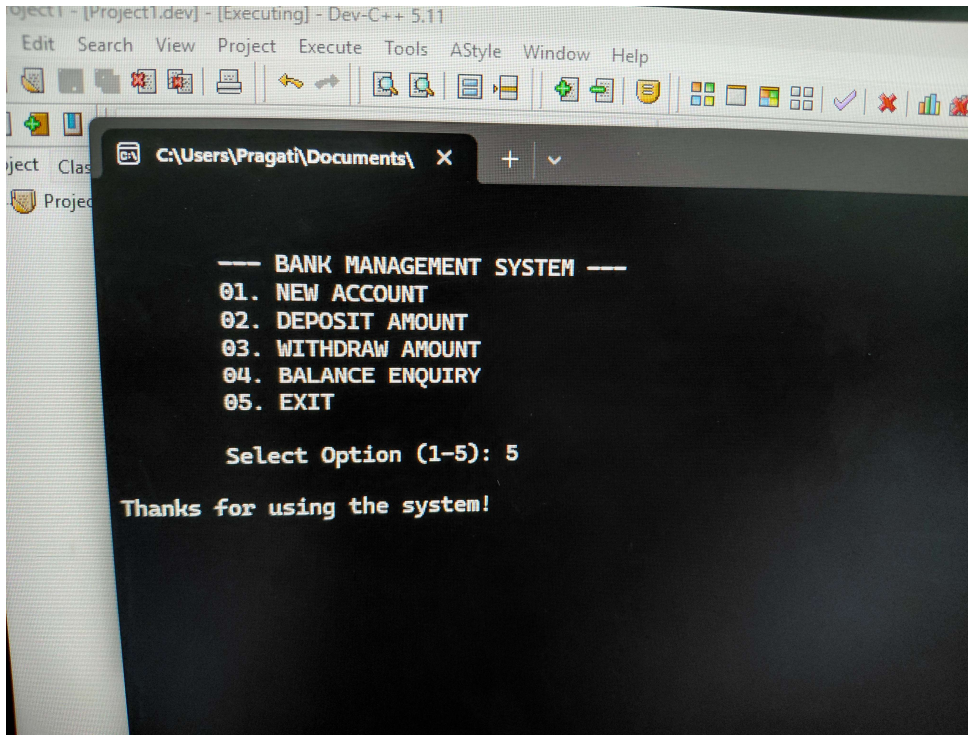


05 | METHODOLOGY



Git Workflow: `git checkout -b feature/X` → `git commit` → `git push` → Pull Request → Merge to main

06 | RESULT



```
Project1 - [Project1.dev] - [Executing] - Dev-C++ 5.11
Edit Search View Project Execute Tools AStyle Window Help
C:\Users\Pragati\Documents\
--- BANK MANAGEMENT SYSTEM ---
01. NEW ACCOUNT
02. DEPOSIT AMOUNT
03. WITHDRAW AMOUNT
04. BALANCE ENQUIRY
05. EXIT

Select Option (1-5): 5

Thanks for using the system!
```

Fig 1: Bank Management System running in Dev-C++ 5.11

Test Results

Feature	Status
New Account	✓ PASS
Login / Auth	✓ PASS
Deposit	✓ PASS
Withdraw	✓ PASS
Balance Enquiry	✓ PASS
Exit Gracefully	✓ PASS



ALL TESTS PASSED — 6 / 6

07 | CONCLUSION

The Banking System project successfully demonstrates how OOP principles can be applied to model a real-world domain in C++.

Each class serves a distinct role:

Account manages balances & history

User handles credentials & ownership

Bank orchestrates all operations

main() is the minimal entry point

All 6 features passed testing. The project is fully managed on GitHub using a clean feature-branch workflow.



OOP Applied



6 Features Built



All Tests Passed



GitHub Managed

This project provides a strong foundation for future development into a production-grade banking application.

08 | FUTURE WORK



File Storage

HIGH

Save data between sessions using file I/O or SQLite database.



Password Hashing

HIGH

Encrypt passwords using SHA-256 or bcrypt for security.



GUI Interface

MEDIUM

Replace console with Qt or web-based front-end.



Account Types

MEDIUM

Add savings, checking, and fixed deposit account types.



Loan Module

LOW

Implement loan application and repayment tracking.



REST API Layer

LOW

Expose banking operations via API for mobile or web clients.

THANK YOU

Banking System — C++ Project

Language: C++ IDE: Dev-C++ 5.11 Version Control: GitHub

Department of Computer Science | 2025–2026

Questions & Discussion